

Resolução do Exame Modelo 1 — Paradigmas de Programação (2025/2026)

Parte 1 — Teoria (6 Valores)

Pergunta 1: Classes Abstratas vs Interfaces (1,5 valores)

Resposta: Uma **classe abstrata** é uma superclasse parcial que pode conter **estado (atributos)**, construtores, métodos concretos e métodos abstratos (`abstract`), permitindo herança simples (`extends`). Deve ser utilizada quando existe uma relação forte de parentesco ("É UM") e partilha de estado ou código entre classes correlacionadas.

Uma **interface** é um **contrato puro de comportamento** que define métodos (implicitamente `public abstract`), sem guardar estado de instância nem construtores. Uma classe pode implementar múltiplas interfaces (`implements A, B`). É preferível quando se pretende definir capacidades comuns a classes que não partilham a mesma hierarquia.

```
// Exemplo: Classe Abstrata com estado e interface com contrato
public abstract class VeiculoBase {
    private String codigo;
    public VeiculoBase(String codigo) { this.codigo = codigo; }
    public String getCodigo() { return this.codigo; }
    public abstract double calcularAutonomia();
}

public interface Refrigeracao {
    double getTemperaturaAlvo();
}

public class CamiaoRefrigerado extends VeiculoBase implements Refrigeracao {
    private double temp;
    public CamiaoRefrigerado(String codigo, double temp) {
        super(codigo);
        this.temp = temp;
    }
    @Override public double calcularAutonomia() { return 500.0; }
    @Override public double getTemperaturaAlvo() { return this.temp; }
}
```

Pergunta 2: Passagem de Argumentos em Java (1,5 valores)

Resposta: Java realiza a passagem de parâmetros **EXCLUSIVAMENTE POR VALOR**:

1. **Tipos Primitivos:** É passada uma cópia do valor original. Alterações ao parâmetro no método **não afetam** a variável fora dele.

2. **Tipos de Referência (Objetos):** É passada uma **cópia da referência (endereço na Heap)**. Modificar o estado do objeto através dessa referência **altera o objeto original**, mas reatribuir a variável de referência para um novo objeto (`v = new ...`) **não afeta** a referência original externa.

```
public class TestePassagem {
    public static void alterarPrimitivo(int x) { x = 99; }

    public static void alterarObjeto(RefrigeratedVehicleImpl v) {
        v.setEnabled(false); // Altera o estado do objeto original na Heap!
        v = new RefrigeratedVehicleImpl("V99", ItemType.CLOTHING, 100.0, 50.0); // NAO altera o ex
    }

    public static void main(String[] args) {
        int val = 10;
        alterarPrimitivo(val); // val continua 10
    }
}
```

Pergunta 3: Conversão de Tipos (Casting) e `instanceof` (1,5 valores)

Resposta: **Upcasting** é a conversão implícita e automática de uma subclasse para um tipo de superclasse ou interface. É sempre seguro. **Downcasting** é a conversão explícita de um tipo genérico para uma subclasse específica, com o risco de lançar `ClassCastException` em runtime se o tipo real não for compatível. Para evitar erros, deve ser testado previamente com o operador `instanceof`.

```
public class TesteCasting {
    public static void processar(Vehicle v) {
        if (v instanceof RefrigeratedVehicle) {
            RefrigeratedVehicle rv = (RefrigeratedVehicle) v; // Downcasting seguro
            System.out.println("Max Km: " + rv.getMaxKilometers());
        }
    }
}
```

Pergunta 4: Identidade vs Igualdade (`==` vs `equals()`) e `toString()` (1,5 valores)

Resposta: O operador `==` compara a **identidade de referências** (verificando se duas variáveis apontam para o exato mesmo endereço de memória Heap). O método `equals()` compara a **igualdade lógica de estado** entre dois objetos. O método `toString()` fornece uma representação textual legível da instância.

```
public class VeiculoImpl implements Vehicle {
    private String code;
    private ItemType supplyType;
    private double maxCapacity;

    public VeiculoImpl(String code, ItemType supplyType, double maxCapacity) {
        this.code = code;
        this.supplyType = supplyType;
        this.maxCapacity = maxCapacity;
    }
}
```

```

    }

    @Override public String getCode() { return this.code; }
    @Override public ItemType getSupplyType() { return this.supplyType; }
    @Override public double getMaxCapacity() { return this.maxCapacity; }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || !(obj instanceof Vehicle)) return false;
        Vehicle other = (Vehicle) obj;
        if (this.code == null) return other.getCode() == null;
        return this.code.equals(other.getCode());
    }

    @Override
    public String toString() {
        return "VeiculoImpl[Code=" + this.code + ", Cap=" + this.maxCapacity + "];"
    }
}

```

Parte 2 — Prática (14 Valores)

Pergunta 1a: Classe `RefrigeratedVehicleImpl` (3 valores)

```

public class RefrigeratedVehicleImpl implements RefrigeratedVehicle {
    private String code;
    private ItemType supplyType;
    private double maxCapacity;
    private double maxKilometers;
    private boolean enabled;

    public RefrigeratedVehicleImpl(String code, ItemType supplyType, double maxCapacity, double maxKilometers, boolean enabled) {
        this.code = code;
        this.supplyType = supplyType;
        this.maxCapacity = maxCapacity;
        this.maxKilometers = maxKilometers;
        this.enabled = true; // Habilitado por defeito
    }

    @Override
    public String getCode() { return this.code; }

    @Override
    public ItemType getSupplyType() { return this.supplyType; }

    @Override
    public double getMaxCapacity() { return this.maxCapacity; }

    @Override
    public double getMaxKilometers() { return this.maxKilometers; }

    public boolean isEnabled() { return this.enabled; }
}

```

```

    public void setEnabled(boolean enabled) { this.enabled = enabled; }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || !(obj instanceof Vehicle)) return false;
        Vehicle other = (Vehicle) obj;
        if (this.code == null) return other.getCode() == null;
        return this.code.equals(other.getCode());
    }

    @Override
    public String toString() {
        return "RefrigeratedVehicleImpl[Code=" + this.code + ", Type=" + this.supplyType +
            ", MaxCap=" + this.maxCapacity + ", MaxKm=" + this.maxKilometers +
            ", Enabled=" + this.enabled + "]";
    }
}

```

Pergunta 1b: Método `main` de Teste (2 valores)

```

public class MainTest {
    public static void main(String[] args) {
        System.out.println("=== TESTE DA CLASSE RefrigeratedVehicleImpl ===");

        RefrigeratedVehicleImpl v1 = new RefrigeratedVehicleImpl("V01", ItemType.PERISHABLE_FOOD,
            System.out.println("Codigo: " + v1.getCode());
            System.out.println("Tipo: " + v1.getSupplyType());
            System.out.println("Capacidade: " + v1.getMaxCapacity());
            System.out.println("Max Km: " + v1.getMaxKilometers());
            System.out.println("Enabled por defeito: " + v1.isEnabled());

        v1.setEnabled(false);
        System.out.println("Enabled apos setEnabled(false): " + v1.isEnabled());

        RefrigeratedVehicleImpl v2 = new RefrigeratedVehicleImpl("V01", ItemType.PERISHABLE_FOOD,
        RefrigeratedVehicleImpl v3 = new RefrigeratedVehicleImpl("V99", ItemType.CLOTHING, 500.0,

        System.out.println("v1.equals(v2) [Mesmo codigo V01]: " + v1.equals(v2)); // true
        System.out.println("v1.equals(v3) [Codigo diferente V99]: " + v1.equals(v3)); // false
        System.out.println("toString: " + v1.toString());
    }
}

```

Pergunta 2a e 2b: Métodos de Geração em `StrategyImpl` (9 valores)

```

public class StrategyImpl implements Strategy {

    public boolean hasCollectableContainer(Vehicle vehicle, AidBox aidbox) {
        if (vehicle == null || aidbox == null) return false;
    }
}

```

```

        Container[] containers = aidbox.getContainers();
        if (containers == null) return false;

        for (int i = 0; i < containers.length; i++) {
            Container c = containers[i];
            if (c != null && c.getType() == vehicle.getSupplyType()) {
                Measurement lastM = c.getLastMeasurement();
                if (lastM != null && c.getCapacity() > 0) {
                    if ((lastM.getValue() / c.getCapacity()) > 0.8) {
                        return true;
                    }
                }
            }
        }
        return false;
    }

    public boolean addAidBoxToRoute(Route route, AidBox aidbox, RouteValidator validator) {
        if (route == null || aidbox == null || validator == null) return false;

        if (validator.validate(route, aidbox)) {
            try {
                route.addAidBox(aidbox);
                return true;
            } catch (RouteException e) {
                return false;
            }
        }
        return false;
    }

    @Override
    public Route[] generate(IIInstitution inst, RouteValidator validator) {
        if (inst == null || validator == null) return new Route[0];

        Vehicle[] vehicles = inst.getVehicles();
        AidBox[] aidBoxes = inst.getAidBoxes();

        if (vehicles == null || aidBoxes == null) return new Route[0];

        Route[] tempRoutes = new Route[vehicles.length];
        int routeCount = 0;

        for (int i = 0; i < vehicles.length; i++) {
            Vehicle v = vehicles[i];
            if (v == null) continue;

            Route currentRoute = new RouteImpl(v); // Classe concreta RouteImpl

            for (int j = 0; j < aidBoxes.length; j++) {
                AidBox box = aidBoxes[j];
                if (box != null && hasCollectableContainer(v, box)) {
                    addAidBoxToRoute(currentRoute, box, validator);
                }
            }

            if (currentRoute.getRoute() != null && currentRoute.getRoute().length > 0) {
                tempRoutes[routeCount] = currentRoute;
            }
        }
    }

```

```
        routeCount++;
    }
}

Route[] result = new Route[routeCount];
System.arraycopy(tempRoutes, 0, result, 0, routeCount);
return result;
}
}
```